WaitGroups (Synchronizing Goroutines)

To properly synchronize our Goroutines, Go provides a built-in counter mechanism called a **WaitGroup** (part of the sync package). It allows the **main** function to pause and wait until a specific number of background Goroutines officially announce that they are finished.

1. How to use a WaitGroup

- **Add(1):** Add 1 to the counter before launching a Goroutine.
- **Done():** Subtract 1 from the counter when the Goroutine finishes.
- **Wait():** Block the current code until the counter reaches exactly 0.

```
func main() {

   var wg sync.WaitGroup

   wg.Add(1) // Tell the WaitGroup we are about to launch ONE Goroutine

   // Launch the Goroutine
   go printMessage("Hello from a Goroutine!", &wg)

   // Tell the main function to stop and WAIT here until the counter hits 0
   wg.Wait()

   fmt.Println("Main function is now safely exiting.")
}

// The function must accept the pointer to the WaitGroup
func printMessage(msg string, wg *sync.WaitGroup) {
   // Use `defer` to guarantee `wg.Done()` is called right before the function exits!
   defer wg.Done()

   fmt.Println(msg)
}
```

2. Why pass a Pointer?

- If you pass the WaitGroup by value, Go will make a *copy* of the WaitGroup.
- Our background Goroutine will call Done() on the *copy*, but the WaitGroup in main() will never know.
- The counter in main() will stay at 1 forever, resulting in a **Deadlock**.